

AP157 Output 10 - April 18, 2026

Benedict A. Pangilinan(bapangilinan5@up.edu.ph)

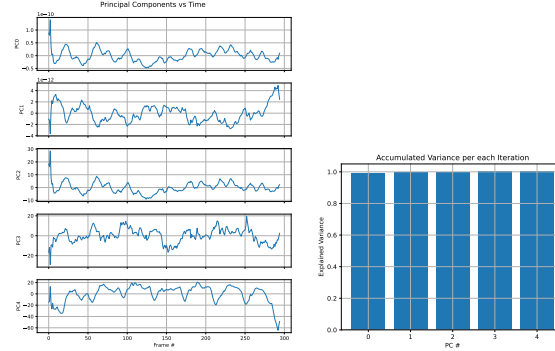


Figure 1: **PCA on a Pendulum** The plots on the left shows the dominant modes of oscillation from the ensemble of observations, while the right shows the accumulated explained variance for each of the first five PCs

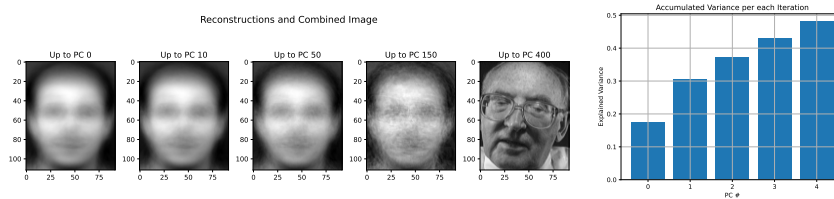


Figure 2: **Image Reconstruction by Principal Component Analysis** The plot on the left depends on the principal components the first figure have shown, but here we superpose it as we reconstruct the image. The right figure shows the explained variance of each of the first 5 PCs

By Fig. 1 we show the principal component analyses of the waves. The author used four videos so the dimensions of the video are $[4, 2, 295]$ where 295 is the number of frames in a video, 2 is the number of features of the video frame. We eventually flatten it such that the features and videos combine, and from there, we conduct the standard algorithm for PCA.

The standard algorithm is taking the covariance matrix, getting its eigenvalues and eigenvectors and taking the dot product of each eigenvectors with the data. For the waveform, this is important due to different observations taking the same observation from different angles, while this may be redundant for this dataset, this would serve essential for learning an image from a set of different ones or identifying dominant modes of a sea surface temperature anomaly data.

If we focus on the right plot of each figure, we can see that the explained variance per (if not cumulative) per principal component varies per the number of features in a sample. If your sample is larger, learning would be slower as such, for the image data, it took close to the full reconstruction before an image is successfully reconstructed. Meanwhile, in the oscillation data, it is satisfactorily reconstructed at first principal component (99.29%)

The difference of image reconstruction with the pendulum

reconstruction is that we restored the mean-centered data to the original coordinates due to the coordinate-dependence of an image. Here is the code for image reconstruction:

```
# Reconstruction function
def reconstruct_with_k(k):
    recons_wtsk = recons_wts[:, :k] # (n_samples, k)
    evecs_k = evec[:, :k] # (n_features, k)
    recons = (recons_wtsk @ evecs_k.T).T # (n_samples, n_features)
    recons += feat_mean.reshape(1, -1)
    return recons

We can thus define a tray of custom restructions (e.g. 0, 50, 100, 150, 400) such that we can reconstruct the image.

# Plot side by side
fig, axes = plt.subplots(1, len(n), figsize=(12, 4))

for i in range(len(n)):
    axes[i].imshow(recons_tray[i], cmap="gray")
    axes[i].set_title(f"Up to PC {n[i]}")
```